

PySpark Documentation Summary

DataFrames:

A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. You can think of a DataFrame like a spreadsheet, a SQL table, or a dictionary of series objects.

Spark DataFrames offer several key advantages for big data processing and analytics:

- **Distributed computing:** Spark distributes data across multiple nodes in a cluster, allowing for parallel processing of big data
- **In-memory processing:** Spark performs computations in memory, which can be significantly faster than disk-based processing
- **Schema flexibility:** Unlike traditional databases, PySpark DataFrames support schema evolution and dynamic typing.
- **Fault tolerance:** PySpark DataFrames are built on top of Resilient Distributed Dataset (RDDs), which are inherently fault-tolerant. Spark automatically handles node failures and data replication, ensuring data reliability and integrity.

Creating a DataFrame

1. From list of Dictionaries

```
[3]: employees = [{"name": "John D.", "age": 30},
                  {"name": "Alice G.", "age": 25},
                  {"name": "Bob T.", "age": 35},
                  {"name": "Eve A.", "age": 28}]

# Create a DataFrame containing the employees data
df = spark.createDataFrame(employees)
df.show()
```

```
+---+-----+
|age|    name|
+---+-----+
| 30| John D.|
| 25| Alice G.|
| 35|  Bob T.|
| 28|  Eve A.|
+---+-----+
```



2. From a Local File

```
[4]: df = spark.read.csv("../data/employees.csv", header=True, inferSchema=True)
df.show()
```

Employee ID	Role	Location
19238	Data Analyst	Seattle, WA
19239	Software Engineer	Seattle, WA
19240	IT Specialist	Seattle, WA
19241	Data Analyst	New York, NY
19242	Recruiter	San Francisco, CA
19243	Product Manager	New York, NY

Or from a local JSON file:

```
[5]: df = spark.read.option("multiline","true").json("../data/employees.json")
df.show()
```

Employee ID	Location	Role
19238	Seattle, WA	Data Analyst
19239	Seattle, WA	Software Engineer
19240	Seattle, WA	IT Specialist
19241	New York, NY	Data Analyst
19242	San Francisco, CA	Recruiter
19243	New York, NY	Product Manager

3. From an existing Dataframe

```
[6]: employees = [
    {"name": "John D.", "age": 30, "department": "HR"},
    {"name": "Alice G.", "age": 25, "department": "Finance"},
    {"name": "Bob T.", "age": 35, "department": "IT"},
    {"name": "Eve A.", "age": 28, "department": "Marketing"}
]
df = spark.createDataFrame(employees)

# Select only the name and age columns
new_df = df.select("name", "age")
```

4. From Table and Database

```
[7]: df = spark.read.table("table_name")
```

From a database

If your table is in a database, you can use JDBC to read the table into a DataFrame.

```
[9]: url = "jdbc:mysql://localhost:3306/mydatabase"
    table = "employees"
    properties = {
        "user": "username",
        "password": "password"
    }

    # Read table into DataFrame
    df = spark.read.jdbc(url=url, table=table, properties=properties)
```

Display the DataFrame

`df.show()` displays a basic visualization of the DataFrame's contents.

`df.show()` has 3 optional arguments: `n`, `truncate`, and `vertical`.

- By default, `df.show()` displays up to the **first 20 rows** of the DataFrame. We can control the number of rows displayed by passing an argument to the `show()` method.
- The `truncate` argument controls the length of displayed column **values** (default value is 20)
- If we set `vertical` to `True`, the DataFrame will be displayed vertically with one line per value

```
[14]: df.show(vertical=True)
```

```
-RECORD 0-----
age | 30
name | John D.
-RECORD 1-----
age | 25
name | Alice G.
-RECORD 2-----
age | 35
name | Bob T.
-RECORD 3-----
age | 28
name | Eve A.
```

DataFrame Manipulation

Rename Columns

Rename DataFrame columns using the `withColumnRenamed()` method:

```
df2 = df.withColumnRenamed("name", "full_name")
df2.show()
```

Filter rows

```
[17]: filtered_df = df.filter((df["age"] > 26) & (df["age"] < 32))
      filtered_df.show()
```

age	name
30	John D.
28	Eve A.

We can also use `df.where` to get the same result:

```
[18]: where_df = df.where((df["age"] > 26) & (df["age"] < 32))
      where_df.show()
```

age	name
30	John D.
28	Eve A.

DataFrames vs. Tables

A DataFrame is an immutable distributed collection of data, only available in the current Spark session.

A table is a persistent data structure that can be accessed across multiple Spark sessions.

A DataFrame can be promoted to a table using the following method:

```
createOrReplaceTempView()
```

This creates a temporary table that can only be accessed within the SparkSession to be able to access this view beyond the session with other session it must be saved to a persistent storage.

Another way is to use global Tem view using `creatGlobalTempView()`

Persistent Storage

In Apache Spark, **persistent storage** refers to saving data or execution state to a non-volatile medium (like cloud storage, a distributed file system, or disk) so it survives beyond the lifespan of a single Spark application, cluster, or memory lifecycle.

DataFrame Data Types

Primitive Data Types

<code>pyspark.sql.types.DataType</code>	Spark SQL Type	Python base equivalent
<code>ByteType</code>	TINYINT	<code>int</code>
<code>ShortType</code>	SMALLINT	<code>int</code>
<code>IntegerType</code>	INT	<code>int</code>
<code>LongType</code>	BIGINT	<code>long</code>
<code>FloatType</code>	FLOAT	<code>float</code>
<code>DoubleType</code>	DOUBLE	<code>float</code>
<code>BooleanType</code>	BOOLEAN	<code>bool</code>
<code>StringType</code>	STRING	<code>string</code>
<code>BinaryType</code>	BINARY	<code>bytearray</code>
<code>TimestampType</code>	TIMESTAMP	<code>datetime.datetime</code>
<code>DateType</code>	DATE	<code>datetime.date</code>

<code>DataType</code>	SQL Type	Python base equivalent	Notes
<code>ArrayType</code>	ARRAY	<code>list</code> (or <code>tuple</code>)	Ordered elements of the same type
<code>MapType</code>	MAP	<code>dict</code>	Keys and values can have different types
<code>StructType</code>	STRUCT	<code>tuple</code> , <code>dict</code> , <code>namedtuple</code>	Named fields with specified types

ByteType Used to store byte-length integers ranging from `-128` to `127`

ShortType Represents a short integer, storing values between `-32768` and `32767`

IntegerType Used to store integer values. Ideal for counts, indices, and any discrete quantity. Python Equivalent: `int` (`-2147483648` to `2147483647`)

LongType Suitable for storing large integer values, Python Equivalent:

`int` (`-9223372036854775808` to `9223372036854775807`)

FloatType Used for floating-point numbers where less precision is acceptable in exchange for performance.

DoubleType Provides double precision floating-point numbers for accurate and precise calculations.

StringType is used for text data; supports Unicode and is capable of storing any string data.

BinaryType is used for raw byte data, such as file contents or images, stored as binary streams.

DateType Used for dates without time, suitable for storing calendar dates like birthdays or specific days. - Python Equivalent: `datetime.date`

TimestampType Stores both date and time, essential for recording precise moments in time, such as log timestamps. Python Equivalent: `datetime.datetime`

Precision for Doubles, Floats, and Decimals

1 . FloatType

It is less precise but requires less storage and can be processed faster than DoubleType. This makes it suitable for applications where a large volume of numerical data needs to be processed quickly and extreme precision is not critical.

2 . DoubleType

It provides a good balance between precision and performance and is suitable for most numerical calculations where precision is important. Ideal for financial calculations where precision is more crucial than computational speed.

3 . DecimalType

Used when dealing with high-precision fixed-scale decimal numbers. The precision and scale can be defined by the user, which makes it invaluable for applications such as financial reporting, where precise decimal representation helps avoid rounding errors. Critical in accounting applications where calculations need to be accurate to the cent.

Complex Data Types in PySpark

PySpark facilitates the handling of nested and structured data, which is essential for working with modern data formats like JSON, XML, and others commonly found in big data ecosystems.

The primary complex data types in PySpark: `ArrayType`, `StructType`, `MapType`

ArrayType Allows storage of multiple values of the same type in a single column. Ideal for data that naturally forms a list, such as tags, categories, or historical data points.

```
array_example = ['apple', 'banana', 'cherry']
```

StructType Enables nesting of DataFrame columns, allowing complex and hierarchical data structures within a single DataFrame cell. Each field in a `StructType` can itself be a complex type.

MapType Represents a key-value pair in a DataFrame column, where each key and value can be of any data type, exactly like python dictionaries.

Semi-Structured Data Processing in PySpark

JSON Processing

JSON is a widely used format in web services and data interchange. PySpark simplifies parsing JSON data into structured DataFrames, making it easy to manipulate and analyze.

Key Functions - `from_json()`: Converts JSON strings into a DataFrame column with a structured data type. - `to_json()`: Converts columns of a DataFrame into JSON strings.

```
[24]: from pyspark.sql.functions import from_json, col
      from pyspark.sql.types import StructType, StructField, StringType, IntegerType

      json_schema = StructType([
          StructField("name", StringType()),
          StructField("age", IntegerType())
      ])

      df = spark.createDataFrame([{"name": "John", "age": 30}, {"name": "Jane", "age": 25}])
      df.select(from_json(col("json_str"), json_schema).alias("parsed_json")).show()

+-----+
|parsed_json|
+-----+
| {John, 30}|
| {Jane, 25}|
+-----+
```

XML Processing

XML is another common format for semi-structured data, used extensively in various enterprise applications. Example: Reading and Processing XML Data

```
[26]: df = spark.read \
      .format('xml') \
      .option('rowTag', 'book') \
      .load('../data/books.xml')
df.select("author", "title", "genre").show()
```

author	title	genre
George Orwell	1984	Dystopian
Jane Austen	Pride and Prejudice	Romance
Mark Twain	Adventures of Huc...	Fiction

VARIANT

The VARIANT data type is a specialized column type designed to store flexible, semi-structured data (like JSON) efficiently without requiring a rigid, pre-defined schema up front. It replaces the legacy practice of storing JSON as plain text STRING or raw STRUCT objects.

Flexibility: VARIANT types can store data structures like JSON or XML without predefined schema constraints, offering high flexibility for data ingestion and manipulation.

Integration: Provides better integration with systems that use semi-structured data, allowing for more direct data exchanges and queries.

Performance: While VARIANT provides flexibility, it might impact performance due to its dynamic nature. It's important to test and optimize data operations involving VARIANT types.

Compatibility: Ensure that all parts of your data pipeline support VARIANT if you're leveraging this data type, especially when exporting data to external systems.

Data manipulation with PySpark

Rename columns

Use `DataFrame.withColumnRenamed` to rename columns.

Drop null values

There are a group of functions in `DataFrameNaFunctions` for missing value handling, we can use `DataFrame.na.drop` or `DataFrame.dropna` to omit rows with `NULL` or `NaN` values.

Fill values

For the remaining missing values, we can use `DataFrame.na.fill` or `DataFrame.fillna` to fill them.

Remove duplicates

To remove duplicates, simply apply `DataFrame.distinct`.

String manipulation

Use `lower()` function to convert all letters to lower case.

Reorder columns

Use `DataFrame.select` for this purpose.

```
[11]: df9 = df7.select("name", "age", "height")
      df9.show()
```

name	age	height
alice	10	80.0
bob	5	80.1
tom	10	80.1
josh	9	78.9
jerry	7	75.3

Transform data

The main part of a data engineering project is transformation.

Select()

```
1 df2= df.select('id','col_2','col_3',
2               sf.sqrt(sf.col("col_4")+sf.col("col_5"))
3               .alias("Sqrt_Col_4_plus_Col_5"))
4 df2.show()
```

id	col_2	col_3	Sqrt_Col_4_plus_Col_5
0	2	3	3.0
1	2	3	3.0
2	2	3	3.0
3	2	3	3.0
4	2	3	3.0
5	2	3	3.0
6	2	3	3.0
7	2	3	3.0
8	2	3	3.0
9	2	3	3.0

Filter rows with where()

The input table may be super huge and contains billions of rows, and we may also be interested in only a small subset.

We can use `where` or `filter` with specified conditions to filter the rows.

```
1 df3 = df2.where(sf.col("id") % 2 == 1)
2 df3.show()
```

id	col_2	col_3	Sqrt_Col_4_plus_Col_5
1	2	3	3.0
3	2	3	3.0
5	2	3	3.0
7	2	3	3.0
9	2	3	3.0

Reshape data using explode()

To make the data easier for aggregation, we can use `explode()` function to reshape the data

name	incomes
alice	[123.0, 456.0, 789.0]
bob	[234.0, 567.0]
tom	[100.0, 200.0, 100.0]
josh	[79.0, 128.0]
bush	[123.0, 145.0, 178.0]
jerry	[111.0, 187.0, 451.0, 188.0, 199.0]

Before Explode Function

name	income
alice	123.0
alice	456.0
alice	789.0
bob	234.0
bob	567.0
tom	100.0
tom	200.0
tom	100.0
josh	79.0
josh	128.0
bush	123.0
bush	145.0
bush	178.0
jerry	111.0
jerry	187.0
jerry	451.0
jerry	188.0
jerry	199.0

After Explode Function

Summarizing via groupBy() and agg()

Use `DataFrame.groupBy(...).agg(...)` to aggregate the data.

```
1 from pyspark.sql import functions as sf
2 df4 = df3.groupBy("name").\
3     agg(sf.avg("income").\
4         alias("Average Income"))
5 df4.show()
```

name	Average Income
BOB	400.5
Tom	133.33333333333334
Alice	456.0
jerry	227.2
josh	103.5
bush	148.66666666666666

Joins

When dealing with multiple dataframe, we likely need to combine them together in some way. The most frequently used approach is joining.

There are seven join methods: INNER, LEFT, RIGHT, FULL, CROSS, LEFTSEMI, LEFTANTI. Default is INNER

SQL on Spark

PySpark offers two main ways to perform SQL operations:

Using `spark.sql()` function to execute SQL queries directly.

```
# Use spark.sql() to select data from a table
spark.sql("SELECT name, age FROM people WHERE age > 21").show()

+-----+----+
|   name|age|
+-----+----+
|Charlie| 30|
+-----+----+
```

Using the PySpark DataFrame API

```
# Read a table using the DataFrame API
people_df = spark.read.table("people")

# Use DataFrame API to select data
people_df.select("name", "age").filter("age > 21").show()

+-----+----+
|   name|age|
+-----+----+
|Charlie| 30|
+-----+----+
```

SQL API: - Ideal for users with SQL backgrounds who are more comfortable writing SQL queries.

DataFrame API: - Preferred by Python developers as it aligns with Python syntax. Provides greater flexibility for complex transformations, especially with user-defined functions (UDFs).

Data loading, storage, file formats

Reading Data

1. Reading CSV Files

```
[3]: csv_df = spark.read.csv("../data/employees.csv", header=True, inferSchema=True)
      csv_df.show()
```

Employee ID	Role	Location
19238	Data Analyst	Seattle, WA
19239	Software Engineer	Seattle, WA
19240	IT Specialist	Seattle, WA
19241	Data Analyst	New York, NY
19242	Recruiter	San Francisco, CA
19243	Product Manager	New York, NY

`header=True`: Treats the first line as column names.

`inferSchema=True`: Automatically infers data types of columns.

2. Reading JSON Files

```
[4]: json_df = spark.read.option("multiline", "true").json("../data/employees.json")
      json_df.show()
```

Employee ID	Location	Role
19238	Seattle, WA	Data Analyst
19239	Seattle, WA	Software Engineer
19240	Seattle, WA	IT Specialist
19241	New York, NY	Data Analyst
19242	San Francisco, CA	Recruiter
19243	New York, NY	Product Manager

`multiline="true"`: Allows reading multi-line JSON structures.

3. Reading Parquet Files

```
[5]: parquet_df = spark.read.parquet("../data/employees.parquet")
parquet_df.show()
```

Employee ID	Location	Role
19239	Seattle, WA	Software Engineer
19243	New York, NY	Product Manager
19242	San Francisco, CA	Recruiter
19241	New York, NY	Data Analyst
19240	Seattle, WA	IT Specialist
19238	Seattle, WA	Data Analyst

Parquet files are highly efficient for storing data due to columnar storage and compression.

4. Reading ORC Files

ORC is another columnar file format, often used in Hadoop environments:

```
[6]: orc_df = spark.read.orc("../data/employees.orc")
orc_df.show()
```

Employee ID	Location	Role
19242	San Francisco, CA	Recruiter
19239	Seattle, WA	Software Engineer
19240	Seattle, WA	IT Specialist
19243	New York, NY	Product Manager
19238	Seattle, WA	Data Analyst
19241	New York, NY	Data Analyst

Writing Data

1. CSV

```
csv_df.write.csv("../data/employees_out.csv", mode="overwrite",
header=True)
```

`mode="overwrite"`: If the directory exists, it will be replaced.

`header=True`: Writes the column names as the first line.

2. Parquet

```
parquet_df.write.parquet("../data/employees_out.parquet",  
mode="overwrite")
```

3. ORC

```
json_df.write.orc("../data/employees_out.orc", mode="overwrite")
```

ANSI Migration Guide - Pandas API on Spark

ANSI mode makes Spark behave more like standard SQL: instead of silently returning unexpected values, Spark will often **throw an exception** when an operation is invalid. It controls how Spark SQL handles **invalid or illegal SQL operations**.

ANSI mode is useful when you want **data quality problems to fail loudly** instead of being silently converted to NULL or otherwise tolerated.

String Number Comparison

ANSI off: Spark implicitly casts numbers and strings, so `1` and `'1'` are considered equal.

ANSI on: behaves like pandas, `1 == '1'` is False.

```
>>> pdf = pd.DataFrame({"int": [1, 2], "str": ["1", "2"]})  
>>> psdf = ps.from_pandas(pdf)  
  
# ANSI on  
>>> spark.conf.set("spark.sql.ansi.enabled", True)  
>>> psdf["int"] == psdf["str"]  
0    False  
1    False  
dtype: bool  
  
# ANSI off  
>>> spark.conf.set("spark.sql.ansi.enabled", False)  
>>> psdf["int"] == psdf["str"]  
0     True  
1     True  
dtype: bool  
  
# Pandas  
>>> pdf["int"] == pdf["str"]  
0    False  
1    False  
dtype: bool
```

Strict Casting

ANSI off: invalid casts (e.g., 'a' → int) quietly became NULL.

ANSI on: the same casts raise errors.

```
>>> pdf = pd.DataFrame({"str": ["a"]})
>>> psdf = ps.from_pandas(pdf)

# ANSI on
>>> spark.conf.set("spark.sql.ansi.enabled", True)
>>> psdf["str"].astype(int)
Traceback (most recent call last):
...
pyspark.errors.exceptions.captured.NumberFormatException: [CAST_INVALID_INPUT]

# ANSI off
>>> spark.conf.set("spark.sql.ansi.enabled", False)
>>> psdf["str"].astype(int)
0    NaN
Name: str, dtype: float64

# Pandas
>>> pdf["str"].astype(int)
Traceback (most recent call last):
...
ValueError: invalid literal for int() with base 10: 'a'
```

Invalid Mixed-Type Operations

ANSI off: Spark implicitly coerces so these operations succeed.

ANSI on: Behaves like pandas, such operations are disallowed and raise errors.

Operation types that show behavior changes under ANSI mode:

- **Decimal-Float Arithmetic:** /, //, *, %
- **Boolean vs. None:** |, &, ^

Related Configurations

spark.sql.ansi.enabled (Spark config)

- Native Spark setting that controls ANSI mode.
- The most powerful config to control both SQL and pandas API behavior.
- If set to **False**, Spark reverts to the old behavior, and the other configs are not effective.

`compute.ansi_mode_support` (Pandas API on Spark option)

- Indicates whether ANSI mode is fully supported.
- Effective only when ANSI is enabled.
- If set to **False**, pandas API on Spark may hit unexpected results or errors.
- Default is **True**.

`compute.fail_on_ansi_mode` (Pandas API on Spark option)

- Controls whether pandas API on Spark fails immediately when ANSI mode is enabled.
- Effective only when ANSI is enabled and `compute.ansi_mode_support` is **False**.
- If set to **False**, forces pandas API on Spark to work with the old behavior even when ANSI is enabled.